

Comprehensive Detailed Reflection on Capstone Experience

The most important things I learnt:

One of the most important lessons was the difference between building a model in theory and creating a usable system. In coursework, I had trained models and evaluated them on accuracy. In this project, I had to take raw data from the MedicationQA dataset and transform it into something that could actually support a chatbot. I learned to check distributions, clean missing values (reducing the dataset from 690 to 617 records), and use visualization tools like histograms and word clouds to understand the structure of the data. This gave me a new appreciation for how data quality directly controls model outcomes.

Another major learning was that in healthcare AI, safety matters more than just scores. At first, I focused on evaluation metrics like ROUGE-L, BLEU, and BERTScore. But I quickly realized that high overlap does not always mean safe or correct answers. By implementing hallucination detection with cosine similarity, I saw how even fluent answers could diverge from reference evidence. Adding the LLM-as-Judge with phi-3-mini allowed me to create structured JSON verdicts that explicitly rated groundedness, hallucination risk, and safety. This changed my thinking: an AI system must be designed to fail safely and avoid harm, especially in medicine.

Finally, I gained deeper technical skills in parameter-efficient fine-tuning. Using LoRA with BioGPT taught me how rank decomposition works at a matrix level and why updating only ~0.36% of weights can still yield strong improvements. This was not just about saving GPU memory, it showed me that efficiency and accuracy can coexist when methods are carefully chosen.

The most impressive thing I achieved was:

The highlight of the project was building a comprehensive medication information chatbot pipeline that didn't just answer questions but also evaluated its own reliability. Instead of stopping at a fine-tuned model, I designed a pipeline that included:

- BioGPT-LoRA as the main domain-adapted model.
- Retrieval grounding using embeddings (MiniLM-L6-v2) over DrugLib and MedQuAD, ensuring answers were evidence-based.
- Cosine similarity hallucination detection that compared generated responses with reference embeddings to quantify factual alignment.

- LLM-as-Judge (phi-3-mini-4k-instruct) that scored every answer on groundedness (0–5), hallucination risk (low/medium/high), and safety (0–5).

The most impressive part of this achievement was the systems-level integration. For example, a typical output showed not only the chatbot’s response but also the latency, hallucination score, reference evidence, and a structured safety verdict. This created transparency for users and evaluators.

What makes me proud is that this was not only a tutorial-following exercise but also I had to design the hallucination detection logic, build the embedding retrieval process, and write the JSON rubric for the judge model myself. It felt like creating a mini production-ready system that could realistically be extended to clinical use.

The things that helped me most were:

Several practices supported my progress:

1. **Systematic documentation:** I maintained notes on each stage (EDA, zero-shot runs, LoRA fine-tuning, hallucination detection). Writing down what I tried and why helped me identify mistakes early, such as poor prompt formatting during zero-shot tests.
2. **Methodical debugging:** When the Mistral-7B model failed to load due to gated access, I avoided wasting time by isolating the issue and pivoting to available models. Similarly, I handled Hugging Face authentication warnings systematically by testing smaller components first.
3. **Synthesizing diverse resources:** Instead of relying on a single tutorial, I drew from academic papers on LoRA, GitHub issue discussions on PEFT, and Hugging Face forum advice on model deployment. This broader approach gave me not just *how* to do things, but also *why* they worked and where they might fail.
4. **Iterative mindset:** My hallucination detection evolved from a simple cosine similarity threshold to a multi-layer approach with a judge model. Each version fixed weaknesses from the previous one, proving that incremental improvements often produce more robust systems than trying for a “perfect” design at once.

I still need to work on the following areas:

Despite progress, I identified areas where I can grow further:

- **Advanced evaluation design:** While I expanded evaluation from 30 to 100 samples, I still need to learn more about designing statistically sound experiments with confidence intervals, hypothesis testing, and power analysis.

- **Scalable deployment skills:** My system ran effectively in Colab, but I have limited experience with deployment to production environments that handle thousands of users, require model serving optimization, or need monitoring and alerts.
- **Deeper domain integration:** My safeguards were primarily technical (similarity scores, LLM judge). I need to practice collaborating with medical experts to refine what counts as “safe enough” for real-world patient information.
- **Architectural innovation:** I successfully applied LoRA to BioGPT, but I want to build confidence in modifying or even designing new architectures (e.g., adjusting attention layers) when standard ones are insufficient.

This unit will help me in the future:

This project gave me a complete view of the AI engineering lifecycle like data analysis, fine-tuning, pipeline integration, evaluation, and reflection. These skills are directly relevant to roles like AI Engineer, ML Researcher, and Technical Project Lead.

The safety mechanisms I implemented will help me in any regulated industry, whether healthcare, finance, or autonomous systems, where reliability and compliance are as important as performance. The project also gave me confidence to work in R&D contexts, since I created novel solutions (hallucination detection, multi-layer judging) rather than only applying existing recipes.

Finally, explaining design trade-offs clearly in my report and code made me better at technical leadership. These skills which guiding decisions, documenting rationale, and communicating complexity simply are transferable to mentoring roles and team-based AI projects.

If I did this unit again I would do the following things differently:

If I repeated this capstone, I would:

- Begin with a formal risk assessment listing possible model access issues, dataset biases, and evaluation challenges. For example, I would have predicted the Mistral-7B gated repo issue and saved time with a backup plan.
- Involve human evaluators earlier, such as medical students or professionals, to provide qualitative judgments about answer safety alongside metrics.
- Use systematic hyperparameter search (e.g., Bayesian optimization) for LoRA instead of relying mainly on literature defaults.
- Explore additional datasets (e.g., DrugBank, MedQuAD full) for broader validation, ensuring the chatbot generalizes beyond MedicationQA.

- Benchmark against stronger baselines, such as commercial medical chatbots, to better position my results in the broader AI landscape.

Other Reflections:

Working under Colab's GPU limits forced me to be creative. LoRA was not just a workaround, it became a better solution than full fine-tuning because it was efficient, flexible, and scalable. This taught me that constraints can spark innovation.

I also realized how interdisciplinary AI development is. Success here required not only machine learning knowledge, but also understanding medical risks, ethical frameworks, and user trust. This reinforced that future AI work must balance technical, social, and regulatory factors.

Most importantly, I recognized the ethical responsibility of building AI systems. Knowing that my chatbot could influence how someone understands medication made me prioritize safety and transparency over flashy results. This project transformed me from a student who fine-tunes models into an AI professional who designs systems for safe real-world impact.